

Regime Navigator: Adaptive Portfolio Construction Under Uncertainty

Goldman Sachs India Hackathon 2026 - Quant Challenge (Asset Management)

Overview

You are a quantitative portfolio manager at a global asset management firm. Your CIO has given you a universe of **100 anonymized securities** and a mandate: construct a dynamic portfolio that delivers consistent risk-adjusted returns across all market environments - bull, bear, crisis, and recovery.

You are **not** given a benchmark to beat, a factor model to follow, or a target return to hit. Your job is to discover structure in the data, extract signal from noise, and build an allocation engine that adapts to changing market conditions - all while managing risk, controlling turnover, and avoiding the trap of overfitting.

Key constraint: You may hold a maximum of **50 securities** at any time. Choosing which securities to include - and which to exclude - is a core part of the challenge.

Your objective: Maximize the **composite score** (defined in the Scoring section). The composite blends risk-adjusted returns, drawdown control, alpha consistency, turnover efficiency, regime adaptivity, and implementation quality. A strategy that excels on training data but collapses on unseen test data will be penalized.

Portfolio Theory Background

This section covers the key concepts you will need. If you are already familiar with portfolio theory, skip ahead to Data Provided.

Risk-Adjusted Returns

- **Sharpe Ratio** = $\text{mean}(\text{daily_returns}) / \text{std}(\text{daily_returns}) * \text{sqrt}(252)$. Measures excess return per unit of total risk. A Sharpe of 1.0 is considered good; above 2.0 is excellent. Negative Sharpe means you are losing money on a risk-adjusted basis.
- **Sortino Ratio** = $\text{mean}(\text{daily_returns}) / \text{downside_std}(\text{daily_returns}) * \text{sqrt}(252)$. Like Sharpe but only penalizes downside volatility (days when returns are negative). A strategy with many small gains and few losses has a higher Sortino than Sharpe.
- **Calmar Ratio** = $\text{annualized_return} / \text{max_drawdown}$. Measures how much return you earn per unit of your worst peak-to-trough loss. Higher is better.

Drawdown

A **drawdown** is the decline from a peak to a subsequent trough in cumulative portfolio value. **Maximum**

drawdown (MaxDD) is the largest such decline. For example, if your portfolio grows from 100 to 120, then falls to 96, your max drawdown is $(120 - 96) / 120 = 20\%$. Drawdowns matter because investors care deeply about how much they can lose, not just how much they can gain.

Diversification and Concentration

The **Herfindahl-Hirschman Index (HHI)** measures portfolio concentration: $HHI = \sum(w_i^2)$. A perfectly equal-weight portfolio of 20 stocks has $HHI = 0.05$; a single-stock portfolio has $HHI = 1.0$. Lower HHI means better diversification. The constraints table caps single-position and sector exposure so that diversification arises naturally from your design rather than from a separate concentration penalty.

Turnover and Transaction Costs

Turnover is the total absolute change in portfolio weights at each rebalance: $\sum(|w_{new} - w_{old}|)$. Each unit of turnover incurs a **transaction cost** (in basis points, where 1 bps = 0.01%). Higher turnover means higher costs, which drag on returns. A strategy that is right but trades too much may score worse than one that is less right but more patient.

Factor Investing

Many quantitative strategies allocate based on **factors** - systematic characteristics that explain asset returns. Common factors include:

- **Value**: Buy cheap (low P/E, low P/B), sell expensive
- **Momentum**: Buy recent winners, sell recent losers
- **Quality**: Buy profitable, low-debt companies
- **Volatility**: Buy low-vol assets for better risk-adjusted returns
- **Size**: Small-cap stocks tend to have higher (but riskier) returns

Factor performance varies by market regime. Momentum can work brilliantly in a bull market but reverse sharply during a crisis. A robust strategy must account for this.

Market Regimes

Financial markets cycle through regimes: **bull** (rising prices, low vol), **bear** (falling prices, rising vol), **crisis** (sharp selloffs, correlation spikes), **recovery** (bounce-back from crisis), and **sideways** (range-bound, low conviction). Observable signals like implied volatility, credit spreads, and sentiment can help identify the current regime - but regime shifts are rarely clean or predictable.

Data Provided

1. Asset Universe - 100 Securities

`asset\_prices.csv` - Daily OHLCV data for 100 synthetic but statistically realistic securities.

Column	Type	Range	Description
date	string	YYYY-MM-DD	Trading date (business days only)
asset_id	string	SEC_001 - SEC_100	Anonymized identifier
open	float	> 0	Opening price

high	float	$\geq \max(\text{open}, \text{close})$	Intraday high
low	float	$\leq \min(\text{open}, \text{close})$	Intraday low
close	float	> 0	Closing price (use this for return calculations)
volume	int	> 0	Daily trading volume in shares; varies by cap class (SMALL caps have lower volume)

Data guarantees:

- All fields are guaranteed non-null for live assets. There are no NaN or missing values in the provided data.
- Some assets may **delist** during the dataset (they simply stop appearing in the price data). Your strategy must handle this gracefully - delisted assets must receive zero weight. Allocating to a delisted asset incurs a 200 bps penalty.
- OHLCV invariants always hold: `high  $\geq$  max(open, close)` and `low  $\leq$  min(open, close)`.

``asset_fundamentals.csv`` - Quarterly fundamental data (released with a 45-day reporting lag).

Column	Type	Typical Range	Description
report_date	string	YYYY-MM-DD	Date data becomes available to you (NOT the quarter-end date). You may only use this data on or after this date.
asset_id	string	SEC_001 - SEC_100	Identifier
sector	string	S01 - S10	Sector classification (stable over time)
market_cap_class	string	LARGE / MID / SMALL	Market capitalization class (stable over time)
pe_ratio	float	5 - 40	Price-to-earnings ratio (trailing 12 months). Lower typically indicates cheaper valuation.
pb_ratio	float	0.5 - 6	Price-to-book ratio. Lower indicates cheaper valuation relative to book value.
roe	float	-5% to 35%	Return on equity. Higher indicates more efficient use of shareholder capital.
debt_equity	float	0 - 2	Debt-to-equity ratio. Higher means more leverage (riskier).
revenue_growth	float	-20% to 40%	Year-over-year revenue growth. Positive indicates growing business.
earnings_surprise	float	-10% to 10%	Actual EPS vs consensus estimate. Positive means beat expectations.
dividend_yield	float	0% - 6%	Indicated annual dividend yield. Higher means more income.
free_cash_flow_yield	float	0% - 12%	Free cash flow / market cap. Higher means more cash generation relative to price.

Critical: Fundamentals have a **45 calendar day reporting lag**. The `report_date` is when the data becomes available, not when the quarter ended. For example, if a quarterly report is released on 2021-08-15, you can only use that data in `allocate()` calls on or after 2021-08-15. Using fundamental data before its `report_date` constitutes look-ahead bias and will result in **disqualification**.

Data guarantees: All fundamental fields are non-null. Fundamentals are published quarterly. The first report becomes available 75 days after the dataset starts (since the first quarter boundary falls after the start date, plus the 45-day lag).

2. Market Environment Indicators

`market\_indicators.csv` - Daily macro and market signals that can help you detect market regimes.

Column	Type	Range	Description
date	string		Trading date
risk_free_3m	float	0 - 5%	3-month risk-free rate (annualized %)
yield_10y	float	0 - 5%	10-year government bond yield (annualized %)
term_spread	float	-200 to +300 bps	Yield curve slope: 10Y minus 3M (in basis points). Negative = inverted curve, often signals recession.
credit_spread_ig	float	50 - 300 bps	Investment-grade corporate bond spread over risk-free. Wider = more credit risk.
credit_spread_hy	float	150 - 900 bps	High-yield (junk bond) spread. Wider = more distress. Spikes during crises.
impl_vol_index	float	8 - 50	Equity implied volatility index (like VIX). Higher = more fear/uncertainty.
realized_vol_20d	float	5 - 50%	Trailing 20-day realized volatility of the asset universe (annualized %).
commodity_index	float	50 - 150	Commodity price basket index (base = 100 at dataset start). Rising = inflationary pressure.
fx_strength_index	float	85 - 115	Trade-weighted currency strength index (base = 100). Higher = stronger domestic currency.
macro_surprise	float	-100 to +100	Economic surprise index. Positive = economic data beating expectations.
sentiment_score	float	-1.0 to +1.0	Market sentiment composite. Positive = bullish, negative = bearish.
liquidity_index	float	0 to 1	Market liquidity proxy. 1 = highly liquid (easy to trade), 0 = severely illiquid (hard to trade).
funding_stress	float	0 to 1	Interbank funding stress indicator. 0 = calm, 1 = severe stress. Spikes during financial crises.

Data guarantees: All indicator fields are non-null. All values are within stated ranges.

3. Data Windows

All data covers **7 non-contiguous windows of exactly 200 trading days each**. There are intentional **gaps** between windows - transition data is withheld.

Open Sample Data (visible to you - use for training, feature engineering, model development):

Window	Period	Trading Days
SAMPLE_A	2021-06-01 to 2022-03-07	200

SAMPLE_A is the only sample window bundled in your candidate pack. It is representative of the structure (200 trading days, 100 assets, weekly rebalance, sector / cap mix, fundamentals lag, indicator panel) of every other window the scorer will run.

Hidden Test Data (used for final evaluation - not provided to you):

There are several hidden test windows of 200 trading days each. Each traverses a distinct sequence of market regimes. The hidden windows are designed to exercise different combinations of persistent bull, persistent bear,

sharp drawdown / crisis, recovery, and sideways consolidation - in proportions and orderings that **do not appear in the sample data**.

Strategies that memorise patterns observed only in SAMPLE_A will not generalise. Strategies that build on durable, regime-agnostic signals - drawn from the price history, the fundamentals, and the macro indicator panel - will perform better on the hidden test windows than strategies tuned to the visible data.

Your Task

Implement the `PortfolioArchitect` class below. The backtesting engine calls `allocate()` every **5 trading days** (weekly rebalance). Your objective is to maximize the **composite score**, computed across the bundled sample window and the hidden test windows. The composite rewards:

1. **High risk-adjusted returns** (Sharpe, Sortino, Calmar ratios)
2. **Drawdown control** (shallower peak-to-trough is better)
3. **Alpha consistency** (stable rolling Sharpe across the window)
4. **Turnover efficiency** (Sharpe per unit of turnover; less trading is rewarded)
5. **Regime adaptivity** (outperforming equal-weight during market stress periods)
6. **Implementation quality** (clear strategy explanation; thoughtful, honest use of the provided data)

There is no benchmark to beat and no target return. Build a strategy that generalizes.

```
import numpy as np
import pandas as pd

class PortfolioArchitect:

    def __init__(self, prices: pd.DataFrame,
                 fundamentals: pd.DataFrame,
                 indicators: pd.DataFrame):
        """
        Called ONCE before backtesting begins.
        You receive ALL training sample data here.
        Fit models, compute signals, pre-process features.

        Args:
            prices: asset_prices.csv for the training slice (sample window)
            fundamentals: asset_fundamentals.csv up to end of the training slice
            indicators: market_indicators.csv for the training slice
        """
        self.n_assets = 100
        self.all_assets = sorted(prices['asset_id'].unique())
        # YOUR INITIALIZATION HERE

    def allocate(self,
                 prices_to_date: pd.DataFrame,
                 fundamentals_to_date: pd.DataFrame,
                 indicators_to_date: pd.DataFrame,
                 current_date: str) -> np.ndarray:
        """
        Called every 5 trading days during the backtest.

        Args:
            prices_to_date: All price data up to current_date
                           (training windows + current test window so far)
            fundamentals_to_date: Fundamentals with report_date <= current_date
            indicators_to_date: Market indicators up to current_date
            current_date: The rebalancing date (YYYY-MM-DD string)

        Returns:
            target_weights: np.ndarray of shape (100,).
        """
```

```

Constraints are enforced automatically
(see Constraints table below).
Weights for delisted assets must be 0.0.

"""
# REPLACE WITH YOUR STRATEGY
# Default: equal weight across all live (non-delisted) assets
latest = prices_to_date[prices_to_date['date'] == current_date]
live = set(latest['asset_id'].values)
w = np.zeros(self.n_assets)
for i, asset in enumerate(self.all_assets):
    if asset in live:
        w[i] = 1.0
total = w.sum()
return w / total if total > 0 else w

```

Constraints (Enforced by Backtester)

All constraints are enforced automatically. If your weights violate a constraint, the backtester will adjust them (clipping, scaling, or zeroing as appropriate) and apply the stated penalty to your returns for that rebalance period.

#	Constraint	Rule	Violation Penalty
1	Max securities	At most 50 non-zero positions	Smallest positions zeroed; 25 bps
2	Max single position	$\text{abs}(w_i) \leq 0.10$ (10%) per asset	Position capped; 50 bps
3	Sector concentration	$\text{Sum abs}(w_i) \leq 0.30$ per sector	Excess scaled down; 40 bps
4	Short limit	Total short exposure ≥ -0.30 (up to 30% short)	Shorts scaled; 50 bps
5	Gross exposure	$\text{Sum abs}(w_i) \leq 1.50$	Hard-capped; 25 bps
6	Net exposure	$0.85 \leq \text{Sum}(w_i) \leq 1.10$ (allows up to 15% cash)	Auto-normalized to 1.0; 25 bps
7	Delisted assets	$w_i = 0$ for delisted assets	Hard-set to zero; 100 bps
8	Near-zero portfolio	If $\text{Sum}(w_i) < 0.01$ after enforcement	All weights set to zero (all-cash)
9	Total penalty cap	Penalties per rebalance capped at 300 bps	Hard cap
10	Runtime	Each window must complete within the per-test-case time limit shown in the HackerRank IDE	Window scored 0

Constraint enforcement order: Constraints are applied in the order listed above. Each step modifies the weights before the next step runs. Penalties from all violated constraints accumulate for that rebalance period, then the total is capped at 300 bps so a single bad rebalance cannot wipe a window.

Note on net exposure: The 0.85-1.10 net-exposure band lets you tilt up to 15% to cash equivalents or take a small leveraged position. Combined with the -30% short cap and 1.50 gross cap, you have meaningful room to express directional + hedged strategies without immediately tripping a penalty.

Transaction Costs

Component	Rate
Base turnover cost	20 bps per unit of turnover: $20 \text{ bps} * \sum(\text{abs}(w_{\text{new}} - w_{\text{old}}))$
SMALL cap impact	+25 bps additional per unit of SMALL cap turnover
MID cap impact	+10 bps additional per unit of MID cap turnover
Minimum trade filter	Weight changes < 0.5% are not executed (filtered out)

How costs are applied: Transaction costs from each rebalance are spread as a daily return drag over the 5-day rebalance period. For example, if a rebalance incurs 40 bps total cost, your daily return is reduced by 8 bps ($40 / 5$) for each of the next 5 trading days.

Example: Suppose you trade 30% of your portfolio (turnover = 0.30) and all traded assets are MID cap. Your cost is: $0.30 * (20 + 10) = 9 \text{ bps}$. This is spread as ~1.8 bps/day drag for 5 days. A full portfolio replacement costs ~80 bps before impact. Frequent trading is expensive.

Scoring

Per-Window Score (0-100)

Each window receives a composite score on the [0, 100] scale. The composite blends the following components:

- **Sharpe Ratio** - return per unit of total volatility
- **Sortino Ratio** - return per unit of downside volatility
- **Calmar Ratio** - return per unit of max drawdown
- **Drawdown Control** - how shallow your worst peak-to-trough was
- **Alpha Stability** - consistency of your rolling Sharpe across the window
- **Turnover Efficiency** - Sharpe per unit of trading activity
- **Regime Adaptivity** - performance during stress periods vs the equal-weight benchmark
- **Implementation Quality** - auto-scored from your submitted source: presence of a strategy-explanation docstring, genuine use of `indicators_to_date` and `fundamentals_to_date`, explicit regime-detection logic, code-length sanity, and a clean anti-cheat inspection

Higher Sharpe / Sortino / Calmar, lower drawdown, lower turnover, steadier rolling Sharpe -> higher window score. The exact normalization ranges and component weights are not published; design for robustness, not for fitting the formula.

Final Score

The final score is a weighted aggregate across all per-window scores, with the hidden test windows carrying the majority of the weight. The sample window earns meaningful but minority credit; out-of-sample performance on the hidden tests dominates the leaderboard.

Leaderboard

Ranked by `FINAL_SCORE` (0 to 100). Ties broken by lower average max drawdown.

What You Are Free To Do

- **Choose any subset of securities:** Pick up to 50 from the 100 available. Your selection strategy is key.
 - **Use any modeling technique:** Factor models, machine learning, deep learning, optimization, Bayesian methods, rule-based, ensemble approaches - your choice.
 - **Engineer any features:** Momentum, mean-reversion, volatility, cross-asset signals, regime indicators, fundamentals - whatever you discover.
 - **Design your own regime detection:** HMM, change-point detection, volatility clustering, indicator thresholds - or skip it entirely.
 - **Choose your risk model:** Sample covariance, shrinkage estimators, factor-based - whatever works.
 - **Go long-only or long-short:** Short positions are allowed (subject to the constraints table) but not required.
 - **Use any allowed library:** `numpy`, `pandas`, `scipy`, `statsmodels`, `scikit-learn` (pinned to 0.24.2 on the HackerRank Python 3 image - sklearn 1.x-only APIs are not available). All model weights and parameters must be embedded in your submitted code. There is no separate model file upload.
-

What You Cannot Do

Rule	Consequence
Look-ahead: access data beyond <code>current_date</code>	Disqualification
Hardcode test weights: date-conditional logic for test periods	Disqualification
LLM APIs at runtime: no calls to language model APIs	Disqualification
Network access at runtime: no internet in sandbox	Error; previous weights kept
Exceed runtime: any test case exceeds the HackerRank IDE time limit	That test case scored 0

General Guidance

1. **Equal-weight (1/N) sets the calibration floor of the leaderboard.** Any well-thought-out portfolio reasoning should improve on it.
 2. **Turnover is expensive.** A strategy that is right but trades too much will score worse than one that is less right but more patient. Watch the Turnover Efficiency component.
 3. **Drawdown control matters enormously.** A strategy with 20 % return and 30 % drawdown scores worse than one with 5 % return and 8 % drawdown.
 4. **Understand your model.** Whether you use 3 signals or 100 features, the key is knowing why your strategy should generalize beyond the training data. Complexity is fine if it is well-understood. The Implementation Quality component rewards explaining your approach clearly.
 5. **Handle missing data and asset lifecycle events** (missing rows, NaN values, assets that disappear from the live universe) **gracefully** - crashing in `allocate()` reverts that rebalance to the previous weights and may cost you score.
-

Files Provided

Your download contains the following files:

File	Purpose
PROBLEM_STATEMENT.md / .pdf	This document
python/solution_template.py	Python 3 starter - implement PortfolioArchitect here
python/solution_equalweight.py	Python 3 working equal-weight reference (baseline)
python/requirements.txt	Python dependencies
java/Solution.java	Java 15 starter - implement init() and allocate() here
java/SolutionEqualWeight.java	Java 15 equal-weight reference
java/README_JAVA.md	How to compile and run Java solutions
local_backtester.py	Lightweight local backtester for the bundled sample window
sample_test_cases/SAMPLE_A/	Bundled sample test case (200-day window)

Each `sample_test_cases/<WINDOW>/` directory contains:

- `asset_prices.csv`, `asset_fundamentals.csv`, `asset_indicators.csv` - data up to end of this window
- `window_config.json` - window dates, rebalance schedule, and asset list

Testing Locally

Before submitting, test your solution against the bundled sample window:

Python:

```
# Run against SAMPLE_A and print weight output
python python/solution_template.py --window-dir sample_test_cases/SAMPLE_A

# Run local backtester (performance metrics)
python local_backtester.py --solution python/solution_template.py

# Run the reference equal-weight solution
python local_backtester.py --solution python/solution_equalweight.py
```

Java:

```
# Compile
javac java/Solution.java

# Run against SAMPLE_A
java -cp java Solution --window-dir sample_test_cases/SAMPLE_A

# Local backtester with Java solution
python local_backtester.py --java-solution java/SolutionEqualWeight.java
```

The local backtester reports: annualised return, volatility, Sharpe, max drawdown, average positions, and turnover. It does **not** use hidden test data or the official scoring formula.

Submission Format

Python 3: Submit your `solution_template.py` (with your `PortfolioArchitect` implementation). The runner harness at the bottom (`main()` function) **must remain intact** - it is how the scoring system calls your code, and it is also what enforces the per-rebalance temporal filtering of inputs. Submissions that modify the runner to bypass this filtering will be treated as look-ahead violations.

Java 15: Submit your `Solution.java` (with your `init()` and `allocate()` implementations). The `main()` method must remain intact.

Your solution reads the multi-section input file from `stdin` (parsed into the `CONFIG`, `PRICES`, `FUND`, `IND` sections by the runner) and writes one CSV row per rebalance date to `stdout`:

```
date, SEC_001, SEC_002, ..., SEC_100
2023-11-21, 0.01000000, ...
```

Allowed libraries (Python): `numpy`, `pandas`, `scipy`, `statsmodels`, `scikit-learn==0.24.2`. The HackerRank Python 3 image does **not** ship `cvxpy` or `torch`, and `scikit-learn` is pinned at `0.24.2` - APIs added in `sklearn 1.x` will raise `ImportError` / `AttributeError` at runtime. **Java:** standard JDK only, no external dependencies.

All model weights and parameters must be embedded in your code. There is no separate model file upload.

Not allowed: LLM APIs, network access at runtime, reading files other than those provided.

Required: Strategy Explanation

Every submission must include a written explanation of the strategy at the top of the submitted file (inside a docstring or block comment, immediately after the module-level docstring). This explanation will be reviewed by a human evaluator to verify that the code matches the stated approach. Submissions without a strategy explanation, or where the explanation is clearly inconsistent with the code, will be disqualified.

Your explanation must cover (in 100-400 words):

1. **Core approach:** What is the high-level strategy (factor model, regime detection, risk parity, etc.)?
2. **Signals used:** Which features from prices, fundamentals, or indicators drive your allocations?
3. **Regime detection logic** (if applicable): How do you identify the current market regime?
4. **Portfolio construction:** How do you convert signals into weights (optimization, ranking, thresholding)?
5. **Key design decisions:** What choices did you make to prevent overfitting or control drawdown?

Example placement (Python):

```
"""
STRATEGY EXPLANATION
=====
Core approach: Volatility-adjusted momentum with drawdown circuit breaker.

Signals used: 20-day and 60-day price momentum normalized by realized volatility
(from market_indicators.csv). PE ratio from fundamentals to exclude expensive assets.

Regime detection: impl_vol_index > 25 triggers defensive mode (shift weight to
low-volatility LARGE-cap assets). Below 25, standard momentum ranking applies.

Portfolio construction: Rank all 100 assets by risk-adjusted momentum score.
Select top 40 (below the 45-position limit). Weight proportional to inverse
volatility, then normalize to sum=1.0.

Key design decisions: 60-day lookback chosen over 20-day to reduce noise. Assets
with pe_ratio > 30 are excluded as potential bubble assets. Minimum trade size
of 1% avoids excessive turnover from small weight drift.
"""
```

Evaluation Pipeline

For each evaluation window (sample + hidden tests):

1. Judge provides data files for this window
2. Your solution reads the files, calls `allocate()` ~40 times, writes weights to stdout
3. Judge reads your weight output
4. Judge simulates portfolio returns using the full returns data for this window
5. Judge enforces constraints and applies penalties
6. Judge computes the per-window composite score (0-100)

Final score = weighted aggregate across all window scores

`allocate()` is called approximately 40 times per window. Keep each call fast (small fraction of the per-test-case time limit shown in the HackerRank IDE) so all 40 calls finish comfortably within budget.

Baseline Calibration

The simplest naive submission - **Equal Weight (1/N)** across all live assets - establishes the floor of the leaderboard. Any submission with non-trivial portfolio reasoning should beat that floor. Hardcoded or date-keyed strategies are auto-disqualified by the anti-cheat inspector and score 0.

Beyond that, no further benchmark scores are published. Verify your own work locally with `local_backtester.py` on the bundled sample window; the hidden test windows are where the leaderboard is decided.

This problem has no single correct answer. The best submissions will demonstrate deep understanding of portfolio theory, creative feature engineering, disciplined risk management, and - most critically - the ability to build strategies that generalize beyond the training sample.